Private Payments to a Username via Payment Codes

Hilawe Semunegus

The buildable-today option for project D3 in dash-ideas-scoping.md, letting anyone pay a DashPay username who is not yet a contact without the recipient exposing every incoming payment. This is the design for the payment-code (BIP47-style) path. The silent-payment research annex and the revised contact-request path are covered in the scoping document.

Plain words. Today you can only pay a DashPay username after you both accept a contact request, because that handshake is where your wallets swap the keys that make fresh addresses. To pay a stranger's username, the recipient has to publish something in advance. If they publish the wrong thing, a plain extended public key, everyone can watch all their incoming payments forever. The fix is to publish a payment code instead, a compact identifier that lets a sender compute a fresh one-off address for each payment while the recipient still advertises only one thing. This is the same idea Bitcoin uses in BIP47, adapted so the small "I am about to pay you" notification rides on Dash Platform instead of the main chain.

The primitive, BIP47 in one paragraph

A payment code encodes a public key and a chain code, effectively a purpose-specific extended public key. To pay, the sender performs a Diffie-Hellman exchange between its own key and the recipient's payment-code key, producing a shared secret, and from that secret derives a fresh destination address unique to this sender-and-recipient pair. Each subsequent payment increments to the next derived address. The recipient, holding the private side, derives the same addresses and watches for funds. No two payments land on the same address, and an outside observer sees only unrelated ordinary payments.

The one thing the sender must do first is tell the recipient to start watching and which payment code to use, so the recipient knows which derivation to scan. In BIP47 that is a notification transaction on the Bitcoin chain. On Dash Platform it becomes a notification document, which is the adaptation below.

The Platform adaptation

Instead of an on-chain notification transaction, the sender writes a notification document to Platform addressed to the recipient's identity, carrying the sender's payment code encrypted to the recipient. Platform credit fees rate-limit spam, and the recipient's wallet finds the notification by querying Platform rather than scanning a chain. The actual payments still happen on the Core chain, to the derived P2PKH addresses, so funds never depend on Platform being available once the notification is delivered.

Resolving the three design questions

Earlier review flagged three questions that separate a sketch from a buildable design. All three have clean answers.

1. Key separation. The payment code must derive from a dedicated key, not the identity's authentication or signing keys. Reusing an identity key across authentication and payment derivation is a key-reuse anti-pattern and would tie payment activity to identity operations. The design publishes a purpose-specific payment-code key at a dedicated derivation path in the DashPay profile, distinct from the identity's keys.
2. Recovery. On a wallet restore from seed, the recipient re-derives its payment-code key from the seed, then queries Platform for all notification documents addressed to it. This rebuilds the full set of watched derivations deterministically. It is strictly easier than BIP47 on Bitcoin, where the recipient must scan the whole chain for notification transactions, because Platform is queryable by index. No notification history is lost on restore, provided the notifications are still on Platform.
3. Graph visibility. The notification document links a sender identity to a recipient identity. Encrypting the payload hides which payment code and the payment details, but the existence of the edge, that identity A notified identity B, may be visible to Platform observers. There are two levels. Version 1 encrypts the payload so observers learn only that a notification occurred, and version 2 uses the shielded state shipping in Platform v4.0 to hide the notification itself. The payment addresses on the Core chain are unlinkable in both versions.

What is buildable today, and what waits

These pieces are buildable on current primitives:

- the payment-code derivation
- the encrypted notification document
- the recipient-side query and scan
- the Core-chain payments to the derived P2PKH addresses

They need a DashPay contract field for the published payment code, a notification document type, and wallet support on both ends.

One piece waits on a Platform feature. The shielded notification (version 2) needs the Orchard shielded state to carry the notification. That is a privacy upgrade, not a blocker for version 1.

Open items

- Confirm the exact DashPay contract schema addition (a payment-code field on the profile document, and a notification document type) against the live DIP-15 contract.
- Decide the notification document's retention and query model, since recovery depends on notifications remaining queryable.
- Security review of the adapted derivation and of the notification-payload encryption.

Where this sits

This is one of four options in project D3 of the scoping document. It is the one buildable on today's primitives, and it serves the pay-a-username use case without waiting on the silent-payment research annex. The originating idea for paying non-contact usernames is Joel Valenzuela's.